Testing

This document describes the testing strategy, test infrastructure, and the programming concepts applied in the test suite.

Overview

The project uses pytest as its test framework. Tests live in the `tests/` directory and cover the database layer, export functionality, email notifications, the Flask dashboard (routes, authentication, roles, the web sign-in page), profile logic, the virtual keyboard, and the fake-data seeding script.

Test configuration is in `pytest.ini`:

```
[pytest]
testpaths = tests
pythonpath = kiosk_signin tests
```

The `pythonpath` setting adds both `kiosk_signin` and `tests` to the Python path, allowing test files to import application modules directly (e.g. `import database`) and shared test helpers.

Running Tests

```
pytest
```

Or for verbose output:

```
pytest -v
```

Running Tests With Coverage

```
pytest --cov --cov-report=term-missing
```

`.coveragerc` (repo root) configures `coverage`/`pytest-cov`:

```
[run]
source = kiosk_signin, dashboard
omit = */tests/*, */__pycache__/*

[report]
show_missing = True
fail_under = 37
exclude_lines =
    pragma: no cover
```

```
    if __name__ == .__main__.:
```

`fail_under` makes the coverage command itself fail (non-zero exit) if coverage drops below the threshold -- this is what CI checks. It's currently set to the project's actual baseline rather than 100%: the Tkinter kiosk screens (`kiosk_signin/screens/*.py`) don't yet have a test harness that drives real widgets, so they sit at 0% coverage and drag the overall number down (`dashboard/app.py` and `database.py` are both in the 90%s on their own). Raise this threshold as screen-level test coverage grows -- see Coverage Gap: Kiosk Screens below. The `exclude_lines` patterns exempt two categories of genuinely untestable code from the count: lines explicitly marked `# pragma: no cover` (used for platform-specific branches like the PyInstaller-frozen `sys.frozen` paths in `database.py`/`dashboard/app.py`, which only execute inside a packaged executable and can't be exercised from a normal test run), and `if __name__ == "__main__":` entry-point guards.

Continuous Integration

`.github/workflows/test.yml` runs the full test suite with coverage on every push and pull request (separate from `.github/workflows/build.yml`, which only builds release binaries when a GitHub release is published -- see 12-build-and-deployment.md). The CI job installs `python3-tk` and `xvfb` and runs pytest under `xvfb-run`, since a couple of tests construct real Tk widgets even without a dedicated screen-test harness.

Test Infrastructure

Database Fixture

The core test fixture is defined in `tests/conftest.py`:

```
@pytest.fixture
def db(tmp_path, monkeypatch):
    monkeypatch.setattr(db_module, "DB_PATH", str(tmp_path / "test.db"))
    db_module.init_db()
    return db_module
```

This fixture:

1. Uses pytest's `tmp_path` fixture to create a temporary directory that is automatically cleaned up after the test.
2. Uses `monkeypatch.setattr()` to redirect the `DB_PATH` constant in `database.py` to a temporary file. This means every test gets a fresh, isolated database.
3. Calls `init_db()` to set up the schema and seed data.
4. Returns the `database` module so tests can call its functions directly.

This approach is a form of dependency injection -- the production database path is replaced with a test-specific one at runtime, without modifying the source code.

Test Helpers

`tests/helpers.py` provides factory functions for creating test data:

```
def member(**overrides):
   defaults = {
       "first_name": "Jane",
       "last_name": "Smith",
       "date_of_birth": "2015-01-01",
       ...
   }
   return {**defaults, **overrides}
```

These use the dictionary merge pattern (`{**defaults, **overrides}`) to provide sensible defaults that can be selectively overridden per test. This keeps tests concise while ensuring all required fields are populated.

Test Modules

---

test_database.py

The largest test module, covering all database operations:

Member CRUD: - Insert, update, search, and delete members. - Verify cascading deletes (deleting a member removes their sign-ins and caregiver). - Test the `person_type` field for member/staff distinction. - Verify that search filters work correctly for each type.

PIN Verification: - Correct PIN returns True. - Incorrect PIN returns False. - Blank PIN allows any input (no restriction).

Sign-in/out: - Recording and retrieving sign events. - Getting the latest sign event (all time and today only). - Getting all of today's signs. - Export queries with date range filtering. - Verifying that member exports exclude staff and vice versa.

Direction Validation: - No signs today: allowed direction is IN. - After sign IN: allowed direction is OUT. - After sign OUT (any time of day): allowed direction is IN again -- no am/pm cutoff and no daily lockout. - Full day sequence test (IN, OUT, IN, OUT) still allows another IN afterwards. - Many IN/OUT cycles in a single day remain unrestricted. - Staff use the exact same unrestricted logic as members.

Admin Password: - Default password works. - Wrong password is rejected. - Password change updates the hash.

Run Groups: - Create a run sheet with members. - Re-saving the same name replaces the member list. - Search by name. - Delete a run sheet.

Sign editing (admin time correction): - `get_sign()` returns the record joined with the member's name; unknown id returns `None`. - `update_sign_time()` changes the stored timestamp for that record only. - `add_sign()` inserts a new record at an explicit timestamp.

Holiday Enrolments: - Seed data includes five holiday periods. - Create, update, and delete enrolments. - Unique constraint prevents duplicates.

Dashboard Users (web dashboard login accounts): - `init_db()` seeds a default `admin`/`1234` account. - Creating a user hashes the PIN; a duplicate name returns `None` instead of raising. - `check_dashboard_login()` accepts the right name/PIN pair and rejects a wrong PIN or unknown name. - Listing returns users ordered by name; deleting removes the row; `set_dashboard_user_pin()` changes which PIN is accepted going forward.

test_dashboard.py

The largest test module by far, testing every Flask dashboard route via Flask's test client. Three fixtures build on each other:

```
@pytest.fixture
def client(db, monkeypatch):
   app.config["WTF_CSRF_ENABLED"] = False   # tested separately, see below
   limiter.enabled = False                  # tested separately, see below
   with app.test_client() as c:
       yield c

@pytest.fixture
def auth_client(client):
   # session["role"] = "admin"
   ...

@pytest.fixture
def staff_client(client, db):
   # creates a real dashboard_users row, session["role"] = "staff"
   ...
```

CSRF and rate limiting are disabled by default in `client` so the rest of the suite can POST plain form data without wiring up real tokens -- they're re-enabled just for the handful of tests that specifically target them (see below).

Tests cover: - Login page rendering; successful and failed login attempts by name + PIN. - Dashboard page loads with analytics. - Logs page pagination and filtering. - Audit log page. - People listing and filtering by type. - Person creation, editing, and deletion. - Export downloads (verifying the response is an Excel file).

Multi-user login, roles, and route protection: - The seeded `admin`/`1234` login redirects to `/dashboard`; a staff login redirects to `/signin` instead. - `session["role"]` and `session["name"]` are set correctly after login. - `/` sends staff to `/signin` and admins to `/dashboard`. - Admin-only routes reject a `staff_client` with an "Admin access required" flash rather than a bare redirect loop or crash. - `/signin` (and the routes it needs) stay reachable for `staff_client`.

Editing a sign-in/out time: `_pair_sign_in_visits` carries the underlying `sign_ins.id` for each side of a visit (`in_id`/`out_id`, `None` for whichever side is missing) so the Logs page can link to editing an individual record. `/logs/sign/<id>/edit` renders the form, accepts a valid `datetime-local` value and updates the record, rejects an invalid one, logs a `SIGN_TIME_EDITED` audit entry, flashes an error on an unknown id, and requires `admin_required`.

Adding a missing sign-in/out time: the same `in_id`/`out_id` split makes it possible to tell exactly which side of a visit is missing, so the Logs page shows an "Add" link (instead of "Edit") for that side. `/logs/sign/add` validates its `member_id` and `direction` params, inserts a new `sign_ins` row via `database.add_sign()` on a valid submission, rejects an invalid timestamp, and logs a `SIGN_TIME_ADDED` audit entry.

Logs search filter: the Logs page's `search` query param filters visits by member/staff name, combinable with the existing status and date filters.

CSRF: tests temporarily re-enable `WTF_CSRF_ENABLED` and confirm a POST without a `csrf_token` field is rejected (400), and that a POST with the token pulled from the rendered form succeeds -- proving the protection is actually wired up end-to-end, not just present in templates.

Web Sign In / Out page (`/signin`) and "Who's Here" (`/status`): - The pages load for both roles but require login for anonymous visitors. - `/signin/search` finds members/staff by name and respects the `type` filter; an empty query returns no results. - `/signin/alerts` returns active alerts scoped to the given member ids, empty when there are none. - Submitting records the sign, respects `get_allowed_direction()` (a person already signed in is skipped rather than recorded again, with the rest of a batch still succeeding), and rejects submissions with no one selected or an invalid direction. - `_parse_signature()` is unit-tested directly: empty input, valid stroke data, a non-list payload, an oversized stroke count, and malformed stroke segments.

Manage Dashboard Users: - Creating a user (with a role) succeeds; a duplicate name is rejected with a friendly flash instead of a raw database error; an invalid role value silently falls back to `"staff"`. - Deleting a user works; deleting the sole remaining admin account is blocked. - The whole page and both actions are admin-only.

Exports page (bulk member export removed, staff export gained a per-person mode): the old "Member Signing Sheets" card and its `/export/members` route are gone -- `test_export_members_route_removed` checks the route now 404s. The "Staff Signing Sheets" card does either bulk ("All Staff") or single-person export depending on whether a `person_id` is present, sharing logic with `/export/person` via `_export_person_response()`.

Staff vs. member `person_type` regression: the `person_form.html` template is shared by both the "new member" and "new staff" screens, switched on a `person_type` variable that's threaded through as a hidden field. A test confirms the hidden field renders with the correct value on the staff-creation form.

test_export.py

Tests the Excel export generation. `generate_export` (members) and `generate_staff_export` (staff) share one underlying layout -- one worksheet per person, one row per sign-in/out session -- so most of the tests below exercise that shared layout through whichever entry point is being tested.

- Member export and staff export each produce a valid .xlsx file, to a file path or to a `BytesIO` buffer.
- Empty data produces a file with a single placeholder sheet rather than an empty/broken workbook.
- Signature rendering produces PNG bytes from stroke data.
- Duration calculation handles various cases (normal, missing start/end, negative).

One worksheet per person (member/staff export unification): member and staff exports used to look different -- members got one sheet per calendar day with a fixed AM/PM grid, staff got a single sheet with a

page-broken section per person. Both were unified onto the same one-sheet-per-person, one-row-per-session layout, driven by the shared `_generate_person_sessions_export()` / `_write_person_sheet()` functions. Covered: - `test_generate_export_one_sheet_per_member` / `test_generate_staff_export_one_sheet_per_staff_member` -- a multi-person export produces one worksheet named after each person, not one shared sheet. - `test_generate_export_multi_day_history_stays_on_one_sheet` -- a member's sign-ins spanning several days all land as separate session rows on their one sheet, rather than being split across per-day sheets. - `test_generate_export_embeds_signature_image` / `test_generate_staff_export_embeds_signature_image` -- a session with a captured signature gets a PNG embedded on the extra row underneath it. - `_unique_sheet_title()` (`test_unique_sheet_title_*`) -- worksheet titles are stripped of Excel's reserved characters (`[ ] : * ? / \`), truncated to 31 characters, and de-duplicated (a second "Alice Brown" becomes "Alice Brown (2)") so two people who'd otherwise collide on a sheet name still each get their own tab.

Session pairing (`_build_staff_sessions`) -- nobody is restricted to one AM and one PM sign, so sessions are built by pairing IN/OUT events chronologically rather than by AM/PM slot (despite the `staff_` in its name, this function is shared by both the member and staff exports): - A single IN/OUT pair produces one session. - Multiple sessions in the same day are all kept, in order (not just the first). - Sessions are grouped correctly across multiple days. - People are sorted by last name. - A dangling IN with no matching OUT yet is kept with `out: None` ("still signed in"). - A stray OUT with nothing open is kept with `in: None` rather than silently dropped. - A second IN while one is already open closes the previous session with no sign-out, then opens a new one (defensive handling -- the kiosk itself never produces this sequence).

Person sheet layout (`_write_person_sheet`, shared by both exports): - A still-open session renders "Still signed in" in the sheet. - The "Total time in the centre" line at the end of a person's sheet sums every session's duration. - The name banner shows the year (or a year range, e.g. "2026–2027", when the exported data spans a calendar year boundary), plus the role for staff. - `_day_month()` formats dates as day/month with no year. - A person with no sessions in the exported range shows "No sign-in records for this period." instead of an empty row list.

test_email_notifier.py

Tests the email notification system using `unittest.mock` to avoid sending real emails:

```
@patch("email_notifier.smtplib.SMTP")
def test_send_notification(mock_smtp, db):
    ...
```

Tests cover: - Email is sent when a caregiver has an email address and notifications are enabled. - Email is not sent when notifications are disabled. - Email is not sent when the caregiver has no email. - Notification preference filtering (IN only, OUT only, BOTH, NONE).

test_profile.py

Tests member profile creation and update logic through the database layer: - Creating a member with all fields. - Updating specific fields. - Caregiver upsert (insert and update).

test_virtual_keyboard.py

Tests the virtual keyboard component without requiring a display: - Key press inserts characters. - Backspace deletes characters. - Shift toggles case. - Shift auto-releases after one character.

test_social_screen.py

A minimal regression guard on `kiosk_signin/screens/social_screen.py`'s source text: asserts the placeholder contact details (website, phone, email) are present and that no real business contact info has crept back in. Reads the module's source directly rather than instantiating it, so it needs no display.

test_seed_fake_data.py

Tests `seed_fake_data.py`, the dummy-data generator used to populate a dev machine's database: - `_staff_day_sessions()` (the varied-time, multi-session staff day generator): every event falls on the requested day, events are chronologically ordered, session count stays within 1-3, at most the final session of a day is ever left open (a simulated forgotten sign-out), directions alternate correctly, and -- checked across many random seeds -- both multi-session days and open (still-signed-in) sessions do actually occur. - `_clear_previous_fake_data()` removes only "Fake"-named people, leaving real records untouched, and is a no-op (doesn't raise) when there's nothing to clear. - Running the seeding helpers twice back-to-back doesn't duplicate fake people (idempotency). - The full `_seed_history()` run actually produces staff days with more than one session -- a regression guard that the dummy data reflects the new unrestricted sign-in/out behaviour rather than the old one-shift-per-day pattern.

Coverage Gap: Kiosk Screens

Unlike the database layer, dashboard, and export system, the Tkinter kiosk screens (`kiosk_signin/screens/*.py`, `kiosk_signin/app.py`, and most of `kiosk_signin/components/widgets.py`) don't currently have automated test coverage -- there's no `tk_root`-style fixture that drives a real Tk event loop, and no `FakeApp` stand-in for asserting on screen-to-screen navigation. This is a pre-existing gap, not something introduced by any specific feature, and it's why `.coveragerc`'s `fail_under` is set to the project's actual baseline rather than 100%.

Building that harness -- fixtures for a live (but off-screen) `Tk` root, a fake `App` object that records `show_screen()` calls instead of actually switching screens, and per-screen test modules exercising search/selection/navigation/validation logic -- is future work, tracked separately from any single feature change. Manual smoke-testing (launching the kiosk and walking through sign-in, run sheets, and admin flows) remains the way to verify kiosk-side changes until that harness exists.

Programming Concepts Applied

Test Isolation

Each test runs with a completely fresh database. The `tmp_path` fixture creates a unique temporary directory per test, and `monkeypatch` redirects the database path. This ensures tests cannot interfere with each other regardless of execution order.

Fixtures and Dependency Injection

pytest fixtures provide test dependencies through function parameters. The `db` fixture encapsulates all database setup, and tests declare their dependency simply by including `db` as a parameter:

```
def test_upsert_member_insert(db):
    mid = db.upsert_member(_member())
    ...
```

This is dependency injection: the test receives its dependencies from the framework rather than creating them itself.

Factory Pattern (Test Helpers)

The `member()`, `caregiver()`, and `staff_member()` helper functions implement the factory pattern. They produce test data with sensible defaults, allowing tests to specify only the values relevant to what they are testing:

```
def test_search_by_name(db):
    mid = db.upsert_member(_member(first_name="Unique", last_name="Person"))
    results = db.search_members("Unique")
    assert any(m["id"] == mid for m in results)
```

Mocking (unittest.mock)

The email tests use `@patch` to replace the SMTP client with a mock object. This allows testing the notification logic without sending real emails or requiring network access:

```
@patch("email_notifier.smtplib.SMTP")
def test_send_notification(mock_smtp, db):
    mock_smtp.return_value.__enter__.return_value = MagicMock()
    ...
```

Arrange-Act-Assert

Tests follow the Arrange-Act-Assert pattern: 1. **Arrange**: Set up test data (create members, sign events, etc.). 2. **Act**: Call the function being tested. 3. **Assert**: Verify the result matches expectations.

Boundary Testing

The direction validation tests specifically target boundary conditions: - The start of day (no signs) -- allowed direction is IN. - Straight after a sign IN -- allowed direction is OUT. - Straight after a sign OUT, at any time of day -- allowed direction is IN again (no am/pm cutoff, no lockout). - A full day sequence from start to finish, and many IN/OUT cycles in a single day.